

```

Ci::Minutes::NamespaceMonthlyUsage.updatecounters(namespaceusage, amountused: 100,
sharedrunnersduration: 100)
  projectusage = Ci::Minutes::ProjectMonthlyUsage.findorcreatecurrent(projectid: project)
  Ci::Minutes::ProjectMonthlyUsage.updatecounters(projectusage, amountused: 100,
sharedrunnersduration: 100)

```

Type :wq to exit the log lines. Do not exit the rails console.

2. Add helper method

```

ruby
def increaseciusage(project:, date:, amountused:, sharedrunnersduration:)
  date = date.utc.beginningofmonth
  projectusage = Ci::Minutes::ProjectMonthlyUsage.where(date:
date).safefindorcreateby(projectid: project.id)
  Ci::Minutes::ProjectMonthlyUsage.updatecounters(projectusage, amountused: amountused,
sharedrunnersduration: sharedrunnersduration)
  rootnamespace = project.rootnamespace
  namespaceusage = Ci::Minutes::NamespaceMonthlyUsage.where(date:
date).safefindorcreateby(namespaceid: rootnamespace.id)
  Ci::Minutes::NamespaceMonthlyUsage.updatecounters(namespaceusage, amountused: amountused,
sharedrunnersduration: sharedrunnersduration)
end

```

3. Use helper method

```

ruby
increaseciusage(project: project, date: 1.month.ago, amountused: 10, sharedrunnersduration:
20)

```

The usage quota page should now reflect the changes data.

</details>

- Secure:
 - Generate project vulnerabilities, execute GITLABQAACCESSTOKEN=XXXXXXXXXX
GITLABURL="https://gitlab.com" bundle exec rake
vulnerabilities:setup[<ProjectId>,<VulnerabilityCount>] --trace from the gitlab/qa directory.
Replace the placeholders in the script with your local access token, project ID, and desired
number of vulnerabilities. An example of this might be GITLABQAACCESSTOKEN=asdfASDF1234-
GITLABURL="http://localhost:3000/" bundle exec rake vulnerabilities:setup[25,10] --trace
 - Populate a merge request with vulnerabilities by following these steps.
- Service Desk: Set up incomingemail, servicedeskemail and MailRoom. These MRs can't be
reviewed on Gitpod and need a working GDK. GDK setup instructions. Video walkthrough.
- Value Stream Analytics: Setup and seed data instructions. Request a developer license as many
VSA features require an EE license.
- Product Analytics: GDK setup instructions. This process can only be done on a local version
of the GDK, not on Gitpod. Additionally, it requires Docker.

If you find yourself struggling with the environment setup, contact the design DRI for assistance.

Review the MR

- Use the checklist
 - Follow the design and UI changes checklist to make sure all main aspects are covered.
 - If changes remain behind a feature flag and a full review in staging is planned, merging before a complete review can be considered. Be cautious as this can lead to unplanned issues.
- Stick to UX requirements
 - Adhere to the UX requirements specified in the issue.
 - Use the follow-ups checklist to create issues for any further updates or missing elements.
- Best practices for reviews:
 - Refer to the best practices for everyone and reviewers.
 - Treat the review as a dialogue to build trust and rapport within the team.
- Commenting:
 - Separate each topic into its comment thread to facilitate individual discussions and resolutions. Create threads on the relevant line(s) of code.
 - Clearly communicate what is required from the author to address or resolve your suggestions.
 - Use the Conventional Comment format to convey your intent.
 - For non-mandatory suggestions, mark them as (non-blocking) to indicate they can be resolved within the merge request or as a follow-up.
 - Try the Chrome/Firefox add-on to apply Conventional Comment prefixes.
- Visual feedback:
 - Share annotated screenshots or screen recordings in your comments. This makes issues clear and communication more efficient.
 - Use free apps like CloudApp, Monosnap, or Mac's Screenshot (see how to capture and annotate).
 - Highlight differences between the implementation and the expected result using a Markdown table. Use the template below:

```
<details>
<summary>Differences table template</summary>
```

markdown

This MR	Expected	
-----	-----	
Image/video	Image/video	

```
</details>
```

- Provide constructive feedback:
 - Acknowledge and praise valuable contributions by the author.
 - If there are concerns, consider:
 - Iterating instead of reverting.
 - Educating to collaborate by sharing your context and asking for adaptations.
 - Seeking a second opinion from a quad member, design manager, or other designers.
 - Creating a follow-up issue to address concerns.
 - Blocking the full release of a feature by creating an issue with a list of items that need addressing (example).

Handoff the MR

After review:

- Keep yourself as a reviewer and submit the review with a summary comment, with the appropriate review status selected. For example, if changes are required, select Request changes.
- Create follow-up issues for any outstanding UX concerns that deviate from the Minimal Valuable Change (MVC), labeling them as Deferred UX (details on UX labels).

Follow-up with authors:

- For work outside your specific group, discuss any unclear documentation with the original author. This can be a casual retrospective, synchronous or asynchronous.

Approval:

- Once confident the MR meets all requirements, approve it by clicking the "Approve" button in the merge request widget.
- Follow the responsibility of the reviewer guidelines for handoff.

Performance indicator

The Product Design merge request (MR) review volume is tracked as a Key Performance Indicator (KPI) for the UX department.

SECTION: product/ux/product-designer/mr-reviews/preview-mr.md

Overview

This page shows different ways to preview merge request (MRs) changes.
For the full review process, see Product Designer MR Reviews.

Preview options

You can preview MR changes for the app, documentation, Pajamas, or company handbook using these methods:

- Gitpod (Get started and Help): Cloud environment that provides a ready-to-use app with one click. Works for GitLab instances (using GDK, see below), docs, Pajamas, and handbook.
- GitLab Development Kit (GDK) (Get started and Help): Run GitLab on your local machine. Requires technical knowledge and can break during updates.
- Review Apps (Get started): Unique links created for each MR. Great for documentation but limited for GitLab instances (improvements planned).
- Sync with author: Although we have a bias towards asynchronous communication, sometimes a quick call with the MR author is the most efficient approach.

{{% details summary="Comparison table" %}}

Gitpod (cloud) GDK (local) Review App Sync with author
----- ----- ----- -----
First start Fast (<5 min) · Very slow (30+ min) GitLab (>30 min) Docs (>20 min) Pajamas (~10 min) Handbook (~10 min) Fast (few mins)
Restarts Fast (<2 min) · Depends on your machine · Very fast (secs) Fast (few mins)
Make changes · · · ·
Preview/test · · · ·
Save state[§] · · · N/A
Toggle feature flags · · · ·
Test data for GitLab instances · · · N/A

{{% /details %}}

Contributing

Product Designers should have general knowledge of Git and DevOps flows. Using GitLab in a live environment and making small contributions helps build:

- Empathy for user workflows.
- Understanding of the product.
- Skills to make small improvements directly (UI copy, component migration, etc.).

Gitpod

To use Gitpod:

1. Create a free Gitpod account.
1. Connect it to your GitLab account. This happens automatically when launching from GitLab.com, but if it doesn't work, see how to manually connect them.

Resources:

- Launch Gitpod
- Gitpod for the GitLab project (i.e. cloud GDK)
- · Get started with Gitpod
- · Review MRs with Gitpod
- GDK commands (start, stop, update, etc.)
- Preview/change GitLab
- Preview/change docs
- Apply GitLab paid license
- Configure additional GitLab features (runners, feature flags, advanced search, etc.)
- More how-to topics for GDK
- Check out branches
- Commit and push changes
- Prevent auto-deleting workspaces
- Gitpod official documentation

GDK

- One-line installation
- · Review MRs with GDK
- Browse your dev server
- GDK commands (start, stop, update, etc.)
- Preview/change GitLab
- Preview/change docs
- Apply GitLab paid license
- Toggle feature flags
- More how-to topics for GDK

Review Apps

- Use Review Apps in MRs
- Log into Review Apps
- Prevent auto-stopping
- · Toggle feature flags. You can enable feature flags using tools like curl or Postman..

Help

If you get stuck:

1. If using GDK (locally or in Gitpod), try to troubleshoot it yourself first.
Then, don't hesitate to reach out for help (see below).
1. Gitpod help
 1. Ask in the gitpod-gdk Slack channel.
 1. Contact: Marcel van Remmerden
1. GDK help
 1. See getting help.
 1. Ask in the gdk Slack channel.
 1. Contact: Taurie Davis, Sunjung Park

SECTION: product/ux/technical-writing/_index.md

The GitLab Technical Writing team collaborates with developers, product managers, and the community to develop product documentation.

Good documentation meets the evolving needs of GitLab customers, users, and administrators. It educates readers about features and best practices. It enables people to efficiently configure, use, and troubleshoot GitLab. The Technical Writing team manages the docs.gitlab.com site and its content, processes, and tooling.

The documentation roadmap drives our efforts to improve both the content and documentation website. For example, we know that people have trouble finding information on docs.gitlab.com. We have roadmap items and OKRs to replatform the docs site, provide better task-based information, and make content easier to find. These larger projects, completed in addition to feature documentation, provide continual, iterative improvement to the user experience of our documentation.

Anyone can contribute to the documentation. Follow our GitLab documentation guidelines.

About Us

For more information on the team size and team members, see [Meet Our Team](#), filtered by Technical Writing. The roles in our team include:

- Technical Writers at Intermediate, Senior, and Staff levels.
- Technical Writing Managers.
- Fullstack Engineers, Technical Writing.
- A Technical Writing Director.

Contact Us

To contact the entire team in a GitLab issue or MR, use [@gl-docsteam](#).

The team manages general documentation-related and team-specific Slack channels:

- docs: Questions and general discussion about GitLab documentation, and requests by GitLab team members for doc and UI text reviews.
- docs-engineering: Discussion about the Docs website and other engineering projects.
- docs-processes: Discussion about documentation processes.
- docs-tooling: Discussion about documentation tooling.
- docs-site-changes-hugo: Automated messages from the docs-gitlab-com project.
- tw-team: Technical Writing team chat.
- tw-social: Technical Writing team social chat.

Learn GitLab tech writing fundamentals

If you're interested in updating or creating GitLab documentation, see [GitLab Technical Writing Fundamentals](#).

This course is aimed at both GitLab team members and community contributors and includes:

- Guidelines for technical writing
- GitLab style conventions
- Information about internal testing
- Instructions for content types

This course is not required to contribute to docs.gitlab.com. Everybody can contribute!

For suggestions and feedback, see the [feedback issue](#).

Documentation

GitLab documentation is crafted to help users, administrators, and decision-makers learn about GitLab features and to optimally implement and use GitLab to meet their DevOps needs.

The documentation is an essential part of the product. Its source is developed and stored with the product in its respective paths in the

GitLab repositories.

It's published at docs.gitlab.com (offering multiple versions of all product documentation) and at the /help/ path on each GitLab instance's domain, with content for that instance's version.

The documentation is the single source of truth for all product information.

We follow a docs-first methodology

with the goal of creating documentation that is complete, accurate, and easy to use.

The documentation should be easy to browse or search for the information you need, and it should be easy to contribute to the documentation itself.

To get started contributing to the documentation, see

Contribute to the GitLab documentation.

For standards and guidelines, see the style guide and word list.

Responsibilities

Team members are assigned to specific DevOps stage groups. The Technical Writing team is broadly responsible for both developing documentation content and UI text, and helping others while they develop content:

- Maintaining documentation for many engineering projects.
- Occasionally developing new content to meet the needs of the community.
- Reviewing and collaborating on documentation plans, reviewing doc merge requests or recently merged docs, and ensuring that content meets style and language standards.
- Reorganizing, revamping, and authoring improved documentation to ensure completeness and a smooth user experience.
- Collaborating with Product Designers on UI text, such as microcopy, links from the UI to documentation, error messages, and UI element labels.
- Acting as Technical Writing Lead for the monthly release post.

Prioritization

When evaluating work to meet our stakeholders' needs, we prioritize in the following order:

1. Feature work (including documenting new features, and providing guidance on UI text)

1. OKR-related work

1. Docs improvements and backlog issues (including stage lead work, docs technical debt and implementing content topic design)

1. All other tasks (including DocOps tasks)

Processes

The team is responsible for developing and maintaining efficient processes, including:

- Ensuring that processes are in place and being followed to keep the GitLab docs up to date.
- Following and optimizing documentation workflows with Product and Engineering, Documentation Team workflows, and the division of work.

- Triaging doc-related issues.
- Refining the Documentation Style Guide and continuously improving content about GitLab documentation and its contribution process.
- Making it easier for anyone to contribute to the documentation while efficiently handling community contributions to docs.

Style Guide

The Documentation Style Guide provides language and style guidance for the product documentation and release posts.

Any Technical Writer (or other contributor) can make suggestions for documentation style updates or additions by creating an issue or merge request with the `~tw-style` label, and then assigning the issue or MR to the Style Guide DRI. GitLab team members can also use the docs Slack channel.

Use the following searches to track completed style-related issues:

- GitLab project style issues
- GitLab project style MRs
- Technical Writing project style issues

Testing

The Technical Writing team develops and maintains toolkits to test GitLab's documentation (and other technical content) for problems. These toolkits include (but aren't limited) to:

- Text content and writing style: `markdownlint`, `Vale`
- Text formatting: `markdownlint`, `yamllint`
- Link validity: `Lychee`
- File permissions and naming: `lint-doc.sh`

Any contributor can suggest changes to our linting rules or tooling by creating an issue or merge request with the `~tw-testing` label, and then assigning the issue or MR to a Technical Writer in the DocOps group.

Translation and internationalization

Everyone can contribute to the translation of GitLab from English into other languages. To learn more about translation and internationalization at GitLab, visit the [Import and Integrate direction page](#) and [Manage stage Category Direction page](#) on Internationalization. For a step-by-step guide to translation contributions, read [Translating GitLab](#).

The `docs.gitlab.com` site is not included in the community efforts to internationalize GitLab. Discussion on translating documentation into other languages is included in this issue.

Assignments

Technical Writers (TWs) collaborate with their assigned groups. TWs can also be assigned other work.

Some content on docs.gitlab.com is not reviewed by TWs.

Assignments to DevOps Stages and Groups

The designated Technical Writer is the go-to person for their assigned groups. They collaborate with other team members to plan new documentation, edit existing documentation, review any proposed changes to documentation, suggest changes to UI microcopy, and generally partner with subject matter experts (SMEs) in all situations where documentation is required.

{{% product/tech-writing %}}

<!--

To update the table above:

- For tech writer's name per stage, change <https://gitlab.com/gitlab-com/www-gitlab-com/-/blob/master/data/stages.yml> and <https://gitlab.com/gitlab-com/content-sites/handbook/-/blob/main/layouts/shortcodes/tech-writing.html>
- To turn off a stage, set tw: false in <https://gitlab.com/gitlab-com/www-gitlab-com/-/blob/master/data/stages.yml>

Reference: <https://gitlab.com/gitlab-com/www-gitlab-com/mergerequests/24952>

-->

{{% alert title="Note" color="primary" %}}

If you were directed here from a documentation page's metadata:

- The metadata doesn't indicate developer ownership, but is meant to direct you to an appropriate Technical Writer.
- If you are part of a development group and would like to add metadata to documentation pages, create an issue in the TW team tasks project for discussion. Additional discussion is in issue 547.
- If the stage is listed as none, see if there is a DRI or use roulette.

{{% /alert %}}

Technical Writers are encouraged to review and improve documentation of other stages but they aren't required to. When contributing to docs they don't own, they must respect the assigned TW's ownership and ensure to request their review and approval when adding significant changes to their docs.

When a Technical Writer is on PTO, the whole team acts as their backup.

<!-- vale handbook.Spelling = NO -->

Stage leads

{{< alert type="note" >}}

This section outlines a process that we experimented with in Q1 and Q2 of FY2025, and rolled out more widely in Q3 of 2025. This process is subject to change.

{{< /alert >}}

Some technical writers are assigned as stage leads for a given DevOps stage.

Stage	Assigned stage lead
Verify	{{< member-by-name "Lysanne Pinto" >}}
AI-powered	{{< member-by-name "Jon Glassman" >}}
Create	{{< member-by-name "Amy Qualls" >}}
Plan	{{< member-by-name "Marcin S·d·ak-Jakubowski" >}}
Application Security Testing	{{< member-by-name "Russell Dickenson" >}}

Stage leads might work across an entire stage, or a subset of groups in the stage. They support other technical writers assigned to groups in the stage.

Stage leads:

- Assume the same responsibilities as technical writers, but with a more targeted focus on proactively creating and improving documentation for their assigned stage.
- Spend approximately 70% of their time on issues and merge request reviews authored by developers for new features and enhancements for their assigned groups.
- Spend the remainder of their time:
 - Creating and refining content to address documentation needs and gaps for their assigned stage
(for example, writing tutorials and use case-based content, restructuring existing content, and working on the information architecture).
 - Supporting other writers in the stage to contribute to documentation improvements.
- Complete a quarterly planning issue to outline the content gaps and improvements that they aim to address over three milestones
(for example, FY25Q3 Stage lead planning issue: Secure). The planning issue is automatically created and assigned to all Technical Writers in the stage on the 20th of the last month before the start of the quarter.
- Apply the relevant tw-lead label to documentation improvement MRs that they drive or provide input on. This label allows us to track the improvements that come out of the stage lead process as one of our performance indicators (PIs). Tableau chart accessible to GitLab team members only.
- Collaborate with other stage leads on documentation improvements.

Over time, and with fewer groups assigned per stage lead, an aspirational goal is for stage leads to spend 70% of their time on proactive work rather than 30%.

For documentation improvements, stage leads are responsible for creating an issue board to track ongoing and planned documentation enhancements and additions.

DocOps group

DocOps is like DevOps, but for documentation. It's an

approach to help streamline the creation, management, and deployment of documentation.

Some Technical Writers are members of the DocOps group, which is responsible for:

- Maintaining content quality through testing and linting in CI and on your local machine.
- Assisting Docs Engineers with operations tasks when asked, or when those engineers are not online. For example, helping with Pages configuration, deployments, scheduled pipelines, and review apps.
- Updating dependencies for linting tools, and rolling those updates out in upstream documentation projects.

The DocOps group is not responsible for the documentation website's code, infrastructure, or build scripts.

DocOps tasks are prioritized below feature work and OKR-related work.

Participation in the DocOps group is based on team requirements. To express interest in joining, speak to your manager.

Assignments to other projects and subjects

For collaboration in other projects and subjects:

```
| Subject |
Assigned Technical Writer |
|:-----|
|:-----|
| The documentation site | {{<
member-by-name "Diana Logan" >}} |
| The documentation site backend (code, automation) | {{<
member-by-name "Sarah German" >}} |
| The documentation's information architecture | {{< member-by-name "Fiona Neill" >}} {{<
member-by-name "Kati Paizee" >}} {{< member-by-name "Suzanne Selhorn" >}} |
| GitLab Design System ("Pajamas") information under content | {{< member-by-name "Fiona Neill"
>}} {{< member-by-name "Kati Paizee" >}} |
| Style Guide | {{< member-by-name
"Fiona Neill" >}} {{< member-by-name "Kati Paizee" >}} |
| Testing (DocOps/Vale/markdownlint) | {{< member-by-name
"Michael Belton" >}} |
| GitLab Development Kit (GDK) | {{< member-by-name "Ashraf Khamis" >}}, {{< member-by-name
"Achilleas Pipinellis" >}}, {{< member-by-name "Evan Read" >}}, {{< member-by-name "Jon
Glassman" >}}, {{< member-by-name "Lorena Ciutacu" >}}, {{< member-by-name "Marcel Amirault"
>}}, {{< member-by-name "Phillip Wells" >}}, {{< member-by-name "Russell Dickenson" >}} |
```

Content not reviewed by TWs

Technical writers do not review content in:

- The doc/development directory. Any Maintainer can merge docs in the doc/development directory.
The only exception is /doc/development/documentation, where the writers maintain guidelines.
- The doc/solutions directory. This information is created, reviewed, merged, and maintained by

Solutions Architects.

Stable counterparts

The Technical Writing team gets assistance with the docs-gitlab-com project from stable counterparts outside the team.

Subject	Person
:-----	:-----
Backend reviews	Ash McKenzie
Frontend reviews	Paul Gasco-Vaillancourt
Support	Mike Lockhart

```
<!-- vale handbook.Spelling = YES -->
```

Docs site stats

The technical writing team supports a large amount of content.

Page count

The number of pages in the five primary repositories (GitLab, Omnibus, Charts, Operator, and Runner):

Date	of pages	Increase/decrease from previous quarter
:-----	:-----	:-----
Mar 2025	2,533	4 %
Dec 2024	2,442	5 %
Sept 2024	2,328	-5 %
June 2024	2,456	6 %
Mar 2024	2,308	5 %
Dec 2023	2,201	5 %
Sept 2023	2,088	8 %
Jun 2023	1,993	5 %
Mar 2023	1,929	3 %
Dec 2022	1,840	-
Sept 2022	1,785	-
June 2022	1,633	-
Jan 2022	1,562	-
May 2020	1,165	-

Change between May 2020 and March 2025: 1,368 more pages (a 117% increase).

Decrease in September 2024 due to moving the architecture blueprints topics to the handbook. For more information, see issue 279.

Page count by area of the left nav

By the end of March 2025, the page count by area of the left navigation:

Word count

The number of words in these repositories:

Date	Word count	Increase/decrease from previous quarter	
-----	-----	-----	
Mar 2025	3,621,172	7 %	
Dec 2024	3,373,709	6 %	
Sept 2024	3,191,353	-4 %	
June 2024	3,325,823	4 %	
Mar 2024	3,183,647	6 %	
Dec 2023	2,990,400	5 %	
Sept 2023	2,842,399	5 %	
Jun 2023	2,701,888	6 %	
Mar 2023	2,546,466	6 %	
Dec 2022	2,397,335	-	
Oct 2022	2,271,350	-	
June 2022	2,166,052	-	
Jan 2022	2,017,183	-	
May 2020	1,190,371	-	

Change between May 2020 and March 2025: 2,430,801 more words (a 204% increase).

Decrease in September 2024 due to moving the architecture blueprints topics to the handbook.
For more information, see issue 279.

The word count has more than doubled in this timeframe.

Word count by area of the left nav

By the end of March 2025, the word count by area of the left navigation:

Analytics

GitLab Team Members can view docs site metrics on the docs.gitlab.com LookerStudio dashboard.
To view page level analytics, select Page views by month, and add the page URL in the value field. Do not include https:// as part of the value.

To make or suggest changes to the dashboard, open an issue in the Marketing Strategy and Analytics project.

Technical Writer PTO

When Technical Writers take paid time off, the rest of the team provides coverage for them.
These team members may require additional context for requests. Requests are incorporated into the list of stage/group and

feature priorities for their primary groups.

Options for groups to get help when an assigned Technical Writer is on PTO are:

- Reviewer Roulette.
A rouletted Technical Writer can be pinged or assigned to an issue or merge request.
- A request in the docs channel in Slack, where it will be picked up by an available volunteer Technical Writer.
- For help with a specific, time-sensitive, in-progress piece of work, a pre-arranged Technical Writer. The Technical Writer can be pinged on issues or merge requests and begin participating.

If taking extended PTO (one week or more), Technical Writers and Managers should use the Technical Writer coverage issue.

This issue can describe exactly who is providing coverage, for what, and by what means.

Taking PTO

When taking PTO, Technical Writers:

1. Ensure their out-of-office messaging reflects the available mechanisms for coverage.

It's important to keep GitLab.com statuses up-to-date to ensure:

- Reviewer Roulette can make accurate suggestions.
- The TW team can easily see the PTO status of all team members when checking the Roulette dashboard.

1. Send a message in the group Slack channels indicating where to find the available mechanisms. For example:

text

I'm off for the holidays (202y-mm-dd - 202y-mm-dd). For help with documentation while I'm away, see

<https://handbook.gitlab.com/handbook/product/ux/technical-writing/technical-writer-ptu> for ways to get help.

For urgent named time-sensitive task matters, ping named TW.

Merge request queue checks

Before a Technical Writer goes on PTO, the writer or their manager should determine who will check the writer's MR queue. The assigned person should check the queue at least once each day in the GitLab Review Workload Dashboard.

The assigned writer does not need to do the work. When they check the queue, they can:

- Assign the MRs to themselves for review.
- Use Roulette to find other TWs to review.

Regularly scheduled tasks

Along with Technical Writers' normally assigned work, there are recurring tasks that need to be regularly completed:

- Release Post Structural Check: The Technical Writing Lead reviews the content for the release post published at the end of each milestone. See the Release Post Scheduling Handbook page for each milestone's assigned writer.
- Monthly doc version: At the end of each milestone, a Technical Writer creates the monthly version for the docs site. The Technical Writer assigned to this task is the writer who completed the release post structural check for the previous milestone.
- Docs project maintenance tasks: Each month, one Technical Writer is assigned to complete maintenance tasks for the documentation site and its content. This involves creating a new issue using the tw-monthly-tasks template in the technical-writing project to track maintenance work. If additional work beyond what's described in the maintenance issue is required, the Technical Writer creates merge requests and additional issues as needed.

<!-- vale handbook.Spelling = NO -->

Schedule for Docs project maintenance tasks:

- November, 2025: {{< member-by-name "Zach Painter" >}}
- October, 2025: {{< member-by-name "Lysanne Pinto" >}}
- September, 2025: {{< member-by-name "Isaac Durham" >}}
- August, 2025: {{< member-by-name "Lorena Ciutacu" >}}
- July, 2025: {{< member-by-name "Phillip Wells" >}}
- June, 2025: {{< member-by-name "Achilleas Pipinellis" >}}
- May, 2025: {{< member-by-name "Marcel Amirault" >}}
- March, 2025: {{< member-by-name "Brendan Lynch" >}}
- February, 2025: {{< member-by-name "Emily Sahlani" >}}
- January, 2025: {{< member-by-name "Marcin S·d·ak-Jakubowski" >}}
- December, 2024: {{< member-by-name "Roshni Sarangadharan" >}}
- November, 2024: {{< member-by-name "Ryan Lehmann" >}}
- October, 2024: {{< member-by-name "Russell Dickenson" >}}
- September, 2024: {{< member-by-name "Marcel Amirault" >}}

<!-- vale handbook.Spelling = YES -->

Reviews

Technical Writers are assigned to review merge requests that contain documentation changes authored by GitLab team members and community contributors. The reviews are assigned by subject matter according to the Technical Writer assignments to stage groups or other specialties.

Levels of edit

The Technical Writers use the following levels of edit:

Light

- Ensure the pipeline passes and no obvious grammar, spelling, or punctuation errors exist.

Medium

- Ensure the pipeline passes and no grammar, spelling, or punctuation errors exist.
- Ensure the content is clear, discoverable, navigable, and written with the user's perspective in mind.
- Ensure the content meets the guidelines in the Documentation Style Guide.

Heavy

- Ensure the pipeline passes and no grammar, spelling, or punctuation errors exist.
- Ensure the content is clear, discoverable, navigable, and written with the user's perspective in mind.
- Ensure the content meets the guidelines in the Documentation Style Guide.
- Ensure the content conforms to the defined topic types.
- Ensure the content fits well into the larger documentation set.
- For UI text, ensure the content meets the standards defined in the Pajamas Design System and the Technical Writer Word List.

How the writers apply the levels of edit

To balance quality, speed, and resource constraints, the Technical Writers apply different levels of edit to different documentation.

These guidelines are meant to provide general guidance. They aren't set in stone, and they can be overridden on a case-by-case basis.

These items do not receive an edit unless it's specifically requested (and if requested, they receive a light edit):

- In the GitLab repository, the Contribution guidelines (in the /development directory).
- In the GitLab repository, the doc/solutions directory. This information is owned by Solutions Architects.
- In the GitLab repository, the blueprint documentation (everything in the architecture/blueprints directory).

These items receive a light edit:

- Documentation outside of the five main GitLab repositories (GitLab, Charts, Operator, Omnibus, and Runner).
- Deprecations and removals.
- Merge requests authored by other Technical Writers, unless the MR is part of an OKR, or the author requests a more in-depth edit.

These items receive a medium edit:

- Day-to-day product documentation requests:
 - New feature work (from stage groups)
 - Improvements
 - Bug fixes
 - Community contributions

- Release post items

These items receive a heavy edit:

- Topic type restructuring efforts ("CTRT")
- OKR work
- UI text

In all cases, the Technical Writer confirms that an authoritative source has checked the documentation for technical accuracy.

The Technical Writer can serve as that authoritative source if they have the required knowledge or can efficiently perform the necessary verification.

Review workflow

To balance velocity and quality, the Technical Writers use this workflow:

- When a Technical Writer opens a merge request, another Technical Writer must review and merge.
 - The Technical Writer should not approve or merge their own MR. Instead, they should request a review from a peer with Maintainer access. The reviewer merges the MR after the final approval.
 - This requirement aligns with the GitLab Code Review Guidelines and satisfies the GitLab Change Management Policy.
- When anyone else (like a developer, community member, or Support team member) opens a merge request:
 - If the MR contains only documentation changes, the Technical Writer:
 - Reviews the content and offers suggestions.
 - Does not directly make large changes (by applying suggestions or pushing commits) to the author's branch unless they have explicit approval in the MR to do so.

Pushing to a branch can cause hard-to-resolve merge conflicts, and content can be accidentally overwritten.
 - Can use suggestions or commits to make changes themselves only if the writer has agreement from the author to make changes directly to the author's branch.

In these cases, the author must always review the Technical Writer's changes before the writer merges, to help ensure accuracy.
 - Can apply small suggestions using the Apply suggestion feature if an MR is nearly ready to merge.

Writers can fix things like missing punctuation, typos, and pipeline failures without additional review.
- Approves and merges the documentation MR when it is ready.
- If the MR is primarily a code change that also contains a documentation update, the Technical Writer:
 - Offers suggestions for any documentation, UI text, and error message changes, but should not apply any suggestion themselves.

Making any changes to a code MR can cause pipelines to fail as code and specs often need to be updated by the engineer to match technical writing suggestions.
- Approves the MR if the documentation changes are ready to merge.
- Does not merge code MRs. The MR must be merged by an engineer who also reviews the code

change.

- If the MR is primarily a documentation change, but also has a small code change to update a link to match the change, the Technical Writer:
 - Reviews the content using the same workflow as a documentation-only MR.
 - Can merge only if the MR has all required approvals.

For more information on review turnaround times, see Review-response SLO.

Triaging automated group mentions

When a bot or a community contributor mentions either @gl-docsteam or several Technical Writers based on CODEOWNERS, TWs should volunteer to:

1. Scan the MR and either volunteer to review it or determine which TW should review it, following the guidelines in Selecting a reviewer.

1. Then, if the MR:

- Seems ready for review, assign the selected TW as a reviewer.
- Doesn't seem ready for review, leave a comment for the contributor asking them to mention the selected TW when they're ready.

1. Edit the bot's comment and format the team mention as code. For example: Hi @gl-docsteam! Please review this documentation merge request. This removes other TWs from the MR's participants list, and they will no longer receive notifications for it. The to-do notification will be updated to show the username in backticks, so team members working from their to-do list will have a visible hint that the MR has been addressed.

Selecting a reviewer

In most cases, Technical Writers should use the GitLab Review Workload Dashboard to identify someone for a technical writing review. Be sure the page's filter is set to show only Technical Writers and sort by Assign events last 7 days.

To get an available Technical Writer, select Spin the wheel! on the Dashboard page. In the specific cases where the selected Technical Writer already has a lot of assigned reviews or has recently been very busy, you can select Spin the wheel! again to get a different writer.

If you have content that needs a specific assignee, or if you have a merge request for a page that has a DRI (such as the Documentation Style Guide), in those cases you can specifically assign the review to that person.

Determining Technical Writer availability

There are occasions when Technical Writers may be too busy for general team merge request reviews, and need to focus on their groups or other priorities. In those cases, Technical Writers can update their GitLab status by selecting the Busy checkbox and adding the `:redcircle:`, which prevents their name from appearing in the reviewer roulette.

For example, Technical Writers on release duty for a milestone should add the busy indicator to their status for the week preceding the release date, to focus on release posts and other requirements.

In all other cases, while Technical Writers can add (and remove) the busy indicator from their profiles, we ask that the busy indicator be in place for no longer than two days at a time, and be employed no more than once every two weeks. (Noting that the use of the busy indicator during releases doesn't affect this.) If you need more time not participating in the review roulette, be sure to talk to your manager so they can help (which may include additional use of the busy indicator).

Merge rights

The Technical Writing team is given merge rights (through Maintainer access) to GitLab projects as part of their role. Not all developers get Maintainer access, so Technical Writers must use this privilege responsibly.

As Maintainers, Technical Writers must limit what they merge to:

- Documentation, typically in Markdown-formatted files.
- UI text, error messages, and link-related updates in code files, with the approvals of appropriate engineers.

You can skip engineer approval and ask a member of the TW leadership team or @marcel.amirault to approve code changes when:

- The only code changes in a documentation MR are link fixes to match changes to documentation files or anchor names, and
- The pipeline completed successfully.
- Documentation-related tooling and configuration such as linters, and changes to the docs-gitlab-com project. Engineers are available for code review and merges.

In addition, Technical Writers must:

- Never merge an MR with a failed pipeline.
- Ensure that MRs are complete before merging, with appropriate labels and milestones.
- Ensure that the DRI has reviewed and approved the MR.

Onboarding Technical Writers

While the Technical Writer is onboarding, they will be assigned to shadow groups and then start contributing as trainees. Veteran Technical Writers will coach them through the process.

For more information about onboarding phases and tasks, see the Technical Writer onboarding template.

Standups

We have set up Geekbot for our twice weekly standups (at 10:00 AM, Tuesday and Thursday, in your local timezone) and a random weekly question (run on Wednesdays at 12:00 PM).

All members can edit and manage the standups.

To add a new member to the daily standup:

1. Visit the Geekbot dashboard and sign in using your Slack account when asked.
1. Select the Tues/Thurs ping standup and search the member by name in the Add participants area.
1. Give the newly-added member Manage access and select Save in the upper right corner.

To add a new member to the Weekly Wednesday Question standup:

1. Visit the Geekbot dashboard and sign in using your Slack account when asked.
1. Select the Weekly Wednesday Question standup and search the member by name in the Add participants area.
1. Give the newly-added member manage access and select Save in the upper right corner.

As a member of the Technical Writing team, you're encouraged to add your question to the list of random Wednesday questions! To do so:

1. Visit the Weekly Wednesday Questions.
1. Under the Questions section, you can see a "This is a random set of questions" question. Hover over the two arrows on the right, and select Edit.
1. Scroll to the bottom and select Add question.
1. Select Save questions.

Community contribution opportunities

We welcome improvements to content as well as to the development of our documentation website, at <https://docs.gitlab.com>.

For more information about community contributions, see:

- List of available issues
- GitLab Docs repository

Make an urgent content update on docs.gitlab.com

The documentation website is refreshed every hour. On rare occasions, we might have to publish documentation updates a little faster. If you need an urgent update, follow the steps to manually deploy the docs site.

Report a docs website problem or infrastructure issue

Report website bugs or feature requests in the issue list for the GitLab Docs project.

For outages or website availability issues, see Docs site infrastructure.

Related topics

- Documentation workflow
- Set up your local environment
- Documentation site architecture

SECTION: product/ux/technical-writing/hiring.md

As GitLab grows, the Technical Writing team expands, and team members participate in interviews. A defined interview process is important for a consistent hiring process.

Hiring process

General resources:

- Interviewing at GitLab
- Technical Writing Manager job family
- Technical Writing job family

Interviews for Technical Writing

The following people interview the candidate for a Technical Writing role:

- Technical Writer
- Technical Writing Manager
- Product Manager or Customer Support representative
- Technical Writing Senior Manager

The goal is to evaluate the candidate's technical skills and knowledge, such as:

- Tooling experience
- Communication skills
- Technical writing and editing principles
- Understanding of Information architecture
- GitLab values

The candidate must be able to onboard quickly and keep up with the pace of GitLab development, as well as align with the company values.

Interview training

If you are a Technical Writer at GitLab and you wish to be part of the hiring process, complete the interview training.

In the last steps of the training, you will:

- Shadow a Technical Writer interviewer, who is your coach.

- Be shadowed by your coach.

Although you must shadow only two interviews and be shadowed in one interview, you can request to shadow or be shadowed in more interviews until you feel ready to start interviewing yourself.

Recommendations

For every interview:

- Make sure you have a good internet connection.
- Dress professionally and respectfully.
- Make sure your Zoom background is not distracting.
- Silence your phone.
- Avoid conducting interviews in public places or rooms with others present.
- Be punctual. Join the call a few minutes early and wait for the candidate. Admit them to the room on time.
- Be friendly and use an informal tone.

Shadow a TW

When you shadow a Technical Writer, observe how they conduct the interview.

Before shadowing

- Familiarize yourself with Greenhouse.
- Familiarize yourself with the interview questionnaire in Greenhouse.
- Prepare for the interview as if you were conducting it yourself.
- Sync with your coach before the interview to ask any questions.

While shadowing

- Observe how your coach approaches the candidate and initiates the interview.
 - Observe how your coach asks questions, politely keeps to the purpose of the interview, and tracks time.
 - Take notes. You and your coach will write up the interview report and submit your scorecards.
- During this phase of shadowing, only your coach's vote counts for the final result.

After shadowing

- Sync again with your coach to discuss the interview.
- Ask any questions you might have.

Interview process

When you are being shadowed by your coach, and during any following interviews:

- Familiarize yourself with the interview questionnaire and the scorecard. Some people schedule about 45 minutes for preparation.
- Test your Zoom link.

- Read the candidate's resume, pre-interview questions, and writing samples.
- Collect questions about their past experiences that might have prepared them to perform well at GitLab.
- Create a Google Doc for each candidate. Add the interview questions, any instructions to yourself, and your raw notes. You don't need to share this document with anyone.

During the interview

Introductions

- Start the interview by introducing yourself. Present the interview more like a conversation than a formal process to help ease any stress.
- Ask if they have any questions about the interview before you start. Tell them that you'll leave time at the end if they have further questions about GitLab, about the team, or anything related to the role.

Tone

- Be gentle, kind, friendly, humble, and respectful.
- Be fairly neutral. Try not to agree or disagree with their responses.
- Say please and thanks. "Could you please tell me about xxx..." and "thank you for sharing."

Questions

- Try to read between the lines of their responses. Sometimes the answer isn't in what they said, but in what they didn't say.
- Ask follow-up questions as needed.
- Identify responses to flag, such as inconsistencies, inaccuracies, and discordances with GitLab values.

Time

- Finish on time. If you need a few more minutes, ask them if it's okay.
- Set an alarm if you need reminders to keep you on track.

Wrap up

At the end of the interview, do not give them false expectations.

You don't know how well they will do in other interviews, so don't say anything about how you think they will do in the process.

Be kind. Thank them for their time and interest.

After the interview

Fill out the candidate's scorecard in Greenhouse.

Takeaways

- Describe the key points of the interview. It's like a "TL;DR" note from your entire report.
- Collect positive and negative major points, flags and potential flags, and the overall result

from your perspective.

Notes for the next interviewer

For anything unclear to you, or needs a follow-up, or needs further investigation, add in a note for the next interviewer.

Full notes and results

- Add your polished full notes to Greenhouse.
- Be thoughtful to choose the result ("Strong Yes," "Yes," "No," "Strong No").
- Submit the scorecard.
- Discuss with your coach, if you're being shadowed.

SECTION: product/ux/ux-research/_index.md

{{< youtube "EQ750KX6nU" >}}

Who the UX Research team works with

We work closely with designers, product managers, and developers to ensure that the product meets user needs and expectations. We serve as a bridge between the user and the product team.

Who we research

The UX Research team researches customers, end users, potential users, stakeholders, competitor users, and specialized user groups to ensure the product meets diverse needs and expectations.

How the UX Research team works

Using a range of research methodologies (including user interviews, surveys, usability testing, diary studies, and card sorting) we collect raw data on user behaviors, needs, and preferences.

We then apply rigorous data analysis and synthesis to transform this data into valuable insights that inform and shape the design and development of our products.

Note that UX Researchers are one of the many GitLab Team Members who conduct user research. Other roles, such as Product Managers and Product Designers, frequently conduct research with guidance from the UX Research team. All Research DRIs are supported by our UX Research Operations Coordination team to conduct efficient and high-quality research.

If you are planning to lead a research project, you can get started here.

Our two years vision and strategic pillars

Our vision statement consists of two parts:

- On our impact and scope: We deliver key and timely insights to drive user-centric innovation,

and support and influence decisions from design tactics to product strategy.

- On our methodology and reach: Our team excels in impactful research that balances rigour and adaptability, democratizing research, and providing accurate and actionable insights to decision makers across GitLab, from Product to Field.

We focus on 3 strategic themes:

- Create a structured, accessible and impact-driven UX Research ecosystem for all, this includes:

- Deepen strategic impact of insights
- Establish insight repository
- Streamline research Operations
- Implement global research prioritization
- Cultivate engaged partnerships across the organization, this includes:
 - Communicate & evangelize research
 - Scale research & enable counterparts
 - Collaborate across divisions, and join up insights and share metrics
- Encourage a culture of curiosity and experimentation, this includes:
 - Embrace lean and experimental practices
 - Establish service design as a new function
 - Facilitate workshops for innovation
 - Lead with recommendations

Detailed breakdown of our strategic pillars and our 2 years action plan can be found in the UX Research Vision and Strategy Board (internal access only).

Work with UX Research

Working with us as a research participant or a research panelist

We want to learn from you! We are always looking for a range of professionals to take part in our studies, and would love to have you sign-up. You can read more about what to expect as a research participant, and register your interest in taking part in future research.

What you get in return:

- Influence Over Design and Product Decisions: Your feedback will be heard and considered in future design and product decisions directing anything from feature improvements to product strategy.
- Smoother Experience and Better Product: Your insights will help us shape a product that better fits your day-to-day needs, improving your overall experience and productivity.

Working with us as a GitLab internal team

We'd love to collaborate with you, gather your perspective, and partner with you to engage our customers. If your team interacts with customers and users as part of your daily role or make decisions that influence the customer/user experience of GitLab products, we'd love to hear from you!. Here's how you can partner with us:

- Ensure Your Customers Are Represented in UX Research: You can nominate customers as research

participants by sharing their details in the uxreops channel or direct your customers to the UX Research Panel to register their interest.

- Share what you know through GitLab User Insight Repository: Share what you learn about your customers in the uxresearch channel, or connect with a designated UX researcher for specific stages or themes.
- Identify Areas for Improvement That Need Further Research (unknown): If you notice areas that could benefit from deeper exploration, feel free to mention it in the uxresearch channel or contact the UX researcher for the relevant stage or theme. Providing a related issue for reference would be a bonus!
- Leverage UX Research Insights to Guide Your Work: Stay updated by checking out the uxresearchreports channel for the latest insights. You can also explore Dovetail, our research insights management tool, to search by keywords or watch interview session recordings. You can also consult with the UX researcher for the relevant stage or theme, to be more informed when making design, product or customer engagement decisions.

What you get in return:

- Informed Decision-Making: The UX research team provides valuable insights based on user behavior and feedback, helping internal teams make informed, data-driven decisions throughout the product development process.
- Risk Mitigation: UX research helps identify potential issues early in the development process, allowing internal teams to address these challenges before they escalate, thereby reducing the risk of costly mistakes or misaligned products.

Do you have questions about UX Research?

The UX Research team is here for you! Reach out in the uxresearch Slack channel. Below is a full list of our UX Research team handbook content:

Conducting UX Research at GitLab

- Resources for research DRIs
- When to conduct UX Research
- Defining goals, objectives, and hypotheses
- Problem Validation and methods
- Solution Validation and methods
- Foundational research
- Strategic research at GitLab
- UX Research tools/applications
- UX Cloud Sandbox
- UX research in the Artificial Intelligence (AI) space

Research methods we use at GitLab

- Choosing a methodology
- Surveys
 - Designing your survey
 - Common types of survey questions
 - Survey question scales
 - Sample sizes for surveys

- Piloting your survey
- Qualtrics Tips and Tricks
- How to identify low quality data when designing and reviewing your survey
- Jobs to be Done
- Longitudinal studies
- Diary studies
- Mental modeling studies
- Kano Model for feature prioritization
- User story mapping
- Rapid Iterative Testing and Evaluation (RITE)
- Usability testing
- Usability benchmarking
- Unmoderated usability testing
 - Writing a website usability testing script
- UX Bash
- Preference testing
- Facilitating user interviews
 - Writing a discussion guide for user interviews
- How to create a user persona
- Evaluating navigation
 - Testing navigation: early Solution Validation
 - First click testing for navigation
 - Creating a first click study in Qualtrics
 - RITE testing for navigation
 - Comparative testing for navigation

Finding participants

- How to write an effective screener
 - The Common Screener: an efficient way to screen for multiple studies
- Recruiting participants
- Using the UX Research Google Calendar
- Attending a research event
- Creating and managing a research participant panel

Data and research insights

- Finding existing research
- Collecting useful data
- Using quantitative data to find insights
- Analyzing and synthesizing user research data
- Analyzing survey verbatim
- Research insights
- Documenting research insights in Dovetail
- Product Analytics for UX Research

Templates

- UX research report template ([internal link](#))
- User interview note-taking template ([internal link](#))

- Usability testing script template (internal link)
- Usability testing rainbow analysis chart template (internal link)
- Recommendations alignment Figjam template (internal link)
- Usability benchmarking alignment Figjam template (internal link)
- Auto Time on Task (internal link)

Checklists

- User interviews (internal link)
- Surveys (internal link)
- Usability testing (internal link)

UX Research training

- Interview Carousel - Becoming a better interviewer 15 minutes at a time
- UX Research shadowing
- When to conduct UX Research training (internal LevelUp course)

Resources for UX Researchers

- How the UX Research team operates at GitLab
- UX Researcher pairings
- Research prioritization
- The IP Assignment and when to show it
- How to fill in for a UX Research Operations Coordinator
- UX Research growth and development
- Non-Disclosure Agreements for UX Research
- How to publish a blog post to encourage community contributions on Actionable Insights
- How to Conduct UX Research Workshops

Resources for UX Research Operations Coordinators

- UXR Operations Coordination at GitLab
- Recruitment methods
- Research participation gratuities
- Getting the word out about research insights
- ReOps procurement best practices
- Finding SaaS users
- UX research recruiting email tips

Measures and processes the UX Research team is responsible for

- User Satisfaction (USAT) Survey
- System Usability Scale
 - System Usability Scale responder outreach
- Assessing Category Maturity
 - Category Maturity Scorecards
 - Category Maturity - Competitor Comparison
- Tracking research velocity
- Tracking gold, silver, and bronze UX research projects

- Tracking actionable insights

SECTION: product/ux/ux-research/analyzing-research-data.md

It can be tempting to complete a set of interviews or usability tests and think your research project is done. However, without a process for systematically reviewing and making sense of the data you've collected, you risk leaving valuable insights on the table. The next step is to zoom out and look at your findings through a process called research synthesis.

Synthesis helps us take all of the info we've collected, organize it into patterns and themes, and translate those themes into research insights. At the end of this process, you should have answers to the core research questions you set out to answer and evidence that proves or disproves your hypotheses.

Below are some guidelines for how to set yourself up for successful analysis and synthesis while you're conducting research sessions and after your interviews are complete.

During interviews: Gather data and discuss with your team

While capturing your research data, you'll focus on documenting observations (what you saw the users do, or what problems you saw/heard users experience) and quotes (verbatim of what the user actually said, in quotation marks). You may also want to make notes on early interpretations (what you believe users said or did means) and possible solutions (concrete ways to solve the problems identified).

While the bulk of research synthesis happens after you've finished gathering all your data, analysis can start right away. After every user interview, debrief with your teammates who observed the session about what stood out.

Capturing these insights while the session is fresh in your minds makes the overall process much faster and easier. Discussing observations as a group leads to a more thoughtful analysis, reduces cognitive bias, and helps your team form a strong shared understanding of the problem space you're investigating.

After interviews: Find patterns and themes

Synthesizing user interview data

Affinity diagramming is one way of finding themes in a collection of ideas, quotes, or observations. This method helps you draw out insights from qualitative data quickly and collaboratively. This is traditionally done in person with sticky notes on a large blank wall or whiteboard. At GitLab, we use a tool called Figjam to recreate this experience remotely.

We use Dovetail to capture notes from each participant session and to store the insights which result from synthesizing the data. As you record data from research sessions, please remove any personal identifiable information or PII (such as first and last names). Additionally, Dovetail has the option of allowing you to highlight and tag your notes within the platform.

Aside from Dovetail, Figjam is a helpful tool when you are just learning how to synthesize and

analyze your data.

1. Put your notes into Figjam

Don't just copy your notes word for word into a sticky note in Figjam. Read through them and choose the most salient information to create a sticky note from. Best practice is to use one piece of information per sticky note. Have each contributor take one participant to create sticky notes for. At the end of this step, you should have groups of sticky notes for each participant.

2. Cluster the data into themes

Once all your individual notes are in Figjam, begin by grouping similar stickies together. The themes you use should tie directly to your research hypotheses, research goals, and objectives. You can copy these right into your Figjam board so you can keep them top-of-mind while you group individual findings into broader themes. You might also reference the questions listed in this article to ask yourself and your team throughout the synthesis process.

Here are some potential ways to group findings:

- Equivalence: "This finding is the same as this other finding."
- Association: Around the same area of the experience being analyzed, or "This finding is best considered at the same time as this other finding."
- Hierarchy: Larger thematic trends which several findings support, or "This finding is an example of this other finding."

As your groupings start to come together, add text headings that summarize each cluster of similar stickies.

3. Discuss and revise as needed

Organize and reorganize your findings into meaningful categories. See if you need to adjust any of your themes before moving onto the next step.

You also need to check for possible bias. Try "arguing the other side." In other words, build a case from your research against your key insights to see if they still stand up.

4. Distill findings into insights

Once you're happy with your groupings, distill your findings into actionable or informative insights and put them in your Dovetail project. Insights you uncover should come from multiple sources in your research. Depending on how much research you did, the number of insights you uncover may vary.

Using Dovetail in research synthesis

Dovetail is a powerful tool available to GitLab Team Members which allows you to upload research data and tag to help distill insights. GitLab Team Members can create their own custom

tags or follow the UX Research Team's set of GitLab Global Tags or GitLab Section Tags. More information on the Global Tags and helpful tips on creating custom tags can be found on our Documenting Research Insights in Dovetail handbook page.

SECTION: product/ux/ux-research/analyzing-survey-verbatim.md

Verbatim are comments that are submitted to open-ended questions (e.g., "Is there anything that you'd like to share with us about GitLab's usability?") that are typically asked in a survey where respondents can type in their responses in a free-form format. A verbatim analysis helps us understand different phenomena, like the user experience, by looking for themes across those responses.

The process of analyzing verbatim is usually called "coding", which is not related to writing a computer program in a programming language but describes the process of assigning codes that represent what a verbatim is trying to communicate.

- A code is "a word or phrase that acts as a label for a segment of text".
- Coding "refers to the process of labeling segments of text with the appropriate codes".
- Themes represent groups of codes that tell us something about the user experience.

There are two different approaches to categorizing verbatims.

- Top-down: This is when you take a preset list of categories and assign them to individual verbatim. This approach is best used when you already have a sense of what types of codes and themes you'll observe in verbatims.
- Bottom-up approach: This involves creating categories based on an analysis of each individual data point one by one and grouping them to categories that you then refine until you have codes and themes that meaningfully represent your complete sample of responses. This approach takes more time but it is particularly useful if you don't have a sense of what types of codes and themes you'll observe and/or want to help control for bias in your analysis. The bottom up approach is also sometimes referred to as a grounded theory approach - though we're not using that term here because the steps that we've framed below represent a simplified version of that more academic pursuit.

Analyzing verbatim generally follows the steps below. The exact steps that you take will depend on the scope and goals of your research project.

Steps in Verbatim Analysis

Top-down approach

1. Identify your data by sorting and "cleaning" it
1. Assign final themes
1. Create a summary table

Bottom-up approach

1. Identify your data by sorting and "cleaning" it

1. Familiarize yourself with the data
1. Assign early drafts of codes
1. Recursively refi